JetBot Nano AG

Objekterkennung

1 Einführung

In diesem Abschnitt lernt der JetBot, Hindernisse allein anhand seines Kamerabilds zu erkennen und ihnen selbstständig auszuweichen. Dazu werden zunächst Bilder gesammelt, anschließend wird damit ein neuronales Netz trainiert, das der Roboter schließlich im Fahrbetrieb einsetzt.

Dieses Dokument dient als Begleitmaterial und sollte **parallel** zu den Notebooks geöffnet werden. Diese befinden sich in JupyterLab im Ordner:

`Notebooks/collision_avoidance/datacollection.ipynb`

1.1 Live-Kamerabild anzeigen

Zu Beginn wird die Kamera des JetBots gestartet und ihr Bild live im Notebook angezeigt. So lässt sich beim späteren Sammeln der Daten genau sehen, was der Roboter „sieht“.

Die Kamera wird dabei auf eine Auflösung von **224 × 224 Pixeln** eingestellt. Dies ist genau die Bildgröße, die das neuronale Netz später als Eingabe erwartet. Gleichzeitig bleiben die gespeicherten Bilder dadurch klein, und der Datensatz benötigt wenig Speicherplatz.



Abbildung 1: Böse Tomate

1.2 Bilder sammeln: `free` und `blocked`

Für das Training wird ein Ordner `dataset` mit den beiden Unterordnern `free` und `blocked` angelegt. Über die Buttons **add free** und **add blocked** wird das aktuelle Kamerabild im jeweiligen Ordner gespeichert. Der Zähler neben dem Button zeigt an, wie viele Bilder bereits in der Klasse vorhanden sind.

Die Sicherheitszone

Die Einteilung der Bilder orientiert sich an einer gedachten **Sicherheitszone** um den Roboter: Der JetBot muss sich an seiner aktuellen Position einmal komplett auf der Stelle drehen können, ohne irgendwo anzustoßen oder herunterzufallen.

- `blocked` – Die Sicherheitszone ist verletzt. Ein Hindernis befindet sich direkt vor dem Roboter, oder er steht an einer Kante, an der er herunterfallen könnte. Beispiele: Wand, Stuhlbein, Kiste, Tasche, Tischkante.
- `free` – Der Weg ist frei, der Roboter kann ein Stück geradeaus fahren. Hindernisse dürfen im Bild zu sehen sein, solange sie noch weit genug entfernt sind.

Wie sollten die Bilder aussehen?

Für ein gutes Ergebnis ist die **Vielfalt** der Bilder wichtiger als ihre reine Anzahl. Das neuronale Netz kann nur Situationen erkennen, die es beim Training schon einmal gesehen hat. Daher sollten die Bilder möglichst unterschiedlich sein:

- verschiedene Hindernisse (groß, klein, hell, dunkel),
- verschiedene Blickwinkel und Abstände,
- verschiedene Lichtverhältnisse,
- verschiedene Untergründe (Teppich, Fliesen, Holz),
- Tischkanten und Abgründe als `blocked`.

Wie viele Bilder werden benötigt?

Als Richtwert werden **mindestens 100 Bilder pro Klasse** empfohlen. Für ein zuverlässigeres Verhalten sind 150 bis 300 Bilder pro Klasse sinnvoll. Dabei sollten beide Klassen etwa **gleich viele** Bilder enthalten. Andernfalls neigt das Netz dazu, die häufigere Klasse bevorzugt vorherzusagen.

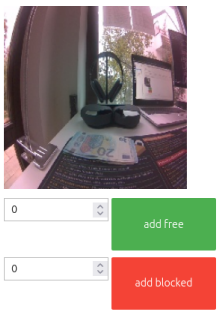


Abbildung 2: add free add blocked

1.3 Daten sammeln

Nun beginnt die eigentliche Datensammlung. Dafür kann einfach der Anleitung im Notebook gefolgt werden: Der JetBot wird per Hand in verschiedene Situationen gestellt und das jeweilige Bild über die Buttons **add free** bzw. **add blocked** gespeichert.

Da es auf Dauer recht mühsam ist, den Roboter für jedes einzelne Foto umzustellen, steht zur Vereinfachung zusätzlicher Notebook-Code zur Verfügung. Mit diesem lässt sich der JetBot per Gamepad steuern, und die Bilder werden direkt per Knopfdruck gespeichert. So kann man mit dem Roboter einfach durch den Raum fahren und deutlich schneller Daten sammeln.

Der Code befindet sich im AG-Repository im Ordner `docs/04_Objekterkennung` und wird im Notebook **nach folgender Zelle** eingefügt:

```
display(image)
display(widgets.HBox([free_count, free_button]))
display(widgets.HBox([blocked_count, blocked_button]))
```

Taste	Funktion
Taste 7	Gas geben
Linker Stick	nach links und rechts lenken
A	Bild als `free` speichern
B	Bild als `blocked` speichern

Nun kann mit dem JetBot durch den Raum gefahren werden. Mit **A** werden Bilder als `free` und mit **B** als `blocked` gespeichert. Die Zähler unter dem Kamerabild zeigen dabei an, wie viele Bilder bereits in jeder Klasse vorhanden sind.

Sind ausreichend Bilder gesammelt, wird zunächst die Steuerung beendet und anschließend im Notebook mit der Zelle `camera.stop()` fortgefahren. Danach wird der gesamte Datensatz mit folgendem Befehl in eine ZIP-Datei gepackt:

```
!zip -r -q dataset.zip dataset
```

Die Datei `dataset.zip` erscheint anschließend im Dateibrowser von JupyterLab und kann per Rechtsklick → *Download* auf dem Laptop gespeichert werden. So existiert eine Sicherungskopie der gesammelten Daten

1.4 Modell trainieren

Als Nächstes werden die gesammelten Bilder verarbeitet und damit das neuronale Netz trainiert. Dazu wird das Notebook der Datensammlung geschlossen und anschließend im selben Ordner die Datei `train_model.ipynb` geöffnet:

`Notebooks/collision_avoidance/train_model.ipynb`

Vor dem Öffnen des neuen Notebooks den Kernel der Datensammlung über *Kernel* → *Shut Down Kernel* beenden. Das Training benötigt den gesamten Arbeitsspeicher des Jetson Nano.

Hinweis

Das Notebook geht davon aus, dass auf einem separaten Rechner trainiert wird, und fordert daher dazu auf, die Datei `dataset.zip` hochzuladen und mit `!unzip -q dataset.zip` zu entpacken. Da hier direkt auf dem JetBot trainiert wird, liegt der Ordner `dataset` bereits im richtigen Verzeichnis. Die Zelle mit `unzip` wird daher **übersprungen**.

1.4.1 Datensatz erstellen

Mit `ImageFolder` werden alle Bilder eingelesen. Die Ordnernamen dienen dabei als Labels und werden alphabetisch durchnummeriert: `blocked` entspricht Klasse 0, `free` Klasse 1.

Jedes Bild durchläuft dabei mehrere Transformationen:

Transformation	Zweck
`ColorJitter`	verändert Helligkeit, Kontrast und Farbe zufällig leicht, damit das Netz robuster gegenüber unterschiedlichem Licht wird (*Data Augmentation*)
`Resize`	bringt alle Bilder auf 224×224 Pixel, die Eingangsgröße des Netzes
`ToTensor`	wandelt das Bild in Zahlenwerte um, mit denen das Netz rechnen kann
`Normalize`	passt die Werte an den Datensatz an, auf dem das Netz ursprünglich trainiert wurde

1.4.2 In Trainings- und Testdaten aufteilen

```
train_dataset, test_dataset = random_split(dataset, [len(dataset) - 50, 50])
```

50 zufällig ausgewählte Bilder werden beiseitegelegt und **nie zum Lernen** verwendet. Mit ihnen wird nach jeder Trainingsrunde geprüft, wie gut das Netz auf unbekannten Bildern funktioniert. Ohne diese Trennung könnte das Netz die Trainingsbilder einfach auswendig lernen, ohne wirklich zu verstehen, was ein Hindernis ist. Dies wird als **Overfitting** bezeichnet.

1.4.3 DataLoader

```
DataLoader(train_dataset, batch_size=8, shuffle=True)
```

Der DataLoader liefert die Bilder in Päckchen (*Batches*) zu je 8 Bildern und mischt sie bei jedem Durchlauf neu. Batches sparen Arbeitsspeicher und machen das Lernen stabiler, da das Netz nicht von jedem einzelnen Bild hin- und hergerissen wird.

1.4.4 Netz laden: Transfer Learning

```
model = models.alexnet(pretrained=True)
model.classifier[6] = torch.nn.Linear(model.classifier[6].in_features, 2)
```

Anstatt ein Netz von null zu trainieren, wird **AlexNet** verwendet. Dieses Netz wurde bereits auf über einer Million Bildern in 1000 Klassen (z. B. Hund, Auto, Stuhl) trainiert. Seine vorderen Schichten haben dabei gelernt, allgemeine Bildmerkmale zu erkennen: zuerst Kanten und Farben, dann Formen und Texturen, schließlich ganze Objektteile. Dieses Wissen ist auch für die Hinderniserkennung nützlich.

Ersetzt wird lediglich die **letzte Schicht**: Statt 1000 Ausgängen besitzt sie nun nur noch 2, nämlich `blocked` und `free`. Dieses Vorgehen wird als **Transfer Learning** bezeichnet. Dadurch reichen bereits einige hundert Bilder aus, während ein Training von null Zehntausende erfordern würde.

1.5 Live-Demo

Im letzten Schritt wird das trainierte Modell auf dem JetBot ausgeführt: Der Roboter wertet jedes Kamerabild in Echtzeit aus und weicht Hindernissen selbstständig aus. Dazu wird im selben Ordner das Notebook `live_demo.ipynb` geöffnet.

Wichtig

Vorher die Kernel aller anderen Notebooks beenden. Die Kamera kann immer nur von einem Notebook gleichzeitig verwendet werden. Andernfalls erscheint die Fehlermeldung `Could not initialize camera`.

1. Modell laden

Zunächst wird dasselbe Netz wie beim Training aufgebaut (AlexNet mit zwei Ausgängen) und anschließend die gelernten Gewichte aus der Datei `best_model.pth` geladen. Das Notebook fordert dazu auf, die Datei hochzuladen. Dies ist nicht nötig, da sie sich bereits im selben Ordner befindet. Danach wird das Modell wieder auf die GPU verschoben.

2. Vorverarbeitung

Die Funktion `preprocess` bringt jedes Kamerabild in dasselbe Format, das das Netz beim Training gesehen hat: Farbreihenfolge anpassen, in Zahlenwerte umwandeln, normalisieren und auf die GPU übertragen.

3. Kamera und Schieberegler

Neben dem Kamerabild werden zwei Regler angezeigt:

- **`blocked`** – zeigt an, mit welcher Wahrscheinlichkeit das Netz gerade ein Hindernis erkennt (0 = frei, 1 = blockiert).
- **`speed`** – legt die Fahrgeschwindigkeit des Roboters fest.

4. Entscheidungslogik

Die Funktion `update` wird für jedes neue Kamerabild aufgerufen. Sie schickt das Bild durch das Netz und berechnet die Wahrscheinlichkeit für `blocked`:

- unter 50 % → der JetBot fährt **geradeaus**,
- ab 50 % → der JetBot dreht sich **nach links**, bis der Weg wieder frei ist.

5. Starten und Stoppen

Mit `camera.observe(update, names='value')` wird die Funktion mit der Kamera verbunden. **Ab diesem Moment kann der Roboter losfahren.** Gestoppt wird er über die Zelle mit `camera.unobserve(...)` und `robot.stop()`. Am Ende wird die Kamera mit `camera.stop()` freigegeben.

Tipp

Den `speed`-Regler zunächst auf 0 lassen und ein Hindernis vor die Kamera halten. Steigt der `blocked`-Balken bei Hindernissen an und sinkt bei freier Sicht, funktioniert das Modell. Erst dann die Geschwindigkeit langsam erhöhen, etwa auf 0.1 bis 0.2.

Modell verbessern

Fährt der JetBot dennoch gegen bestimmte Hindernisse, fehlen von diesen Situationen Bilder im Datensatz. In diesem Fall gezielt weitere Bilder sammeln, das Modell erneut trainieren und wieder testen. Genau dieser Kreislauf aus *Daten sammeln – Trainieren – Testen* ist typisch für das Arbeiten mit künstlicher Intelligenz.

Beispielvideo

Wie das Ergebnis aussehen kann, zeigt folgendes Video. Der JetBot fährt darin selbstständig durch ein Zimmer und weicht dabei Hindernissen aus – ausschließlich anhand des trainierten Modells:

JetBot Collision Avoidance (YouTube)

